Kevin Daniel Sandoval López

Laboratorio 4 Polimorfismo

Análisis

Requisitos funcionales:

La información que el sistema debe de conservar según la categoría:

Equipo debe de conservar: código único, nombre, empresa que representa y puntos acumulados en la tabla de posiciones.

Encuentro debe de conservar: deporte, dos equipos participantes, estado, programación y marcador cuando se haya registrado un resultado válido.

Programación debe de conservar: fecha, hora de inicio entera entre 0 y 23 y sede del evento.

Exhibición debe de conservar: título, estado y programación, no tiene equipos, no tiene marcador ni resultado.

Copa debe de conservar: equipos registrados y eventos creados, tanto encuentros como exhibiciones.

Operaciones con condiciones

1. Registrar equipos con código, nombre y empresa, el código no puede estar vacío ni repetido y los puntos iniciales deben ser cero.

2. Crear encuentros de fútbol sala o baloncesto entre dos equipos registrados y diferentes, inicialmente estando sin programar y sin resultado.

3. Crear exhibiciones con un título, inicialmente estando sin programar, no tendrán equipos, marcador ni resultado.

4. Programar eventos con fecha válida, hora entera entre 0 y 23 y sede disponible, en los encuentros, ninguno de los equipos puede tener otro encuentro en esa misma fecha y hora.

5. Reprogramar eventos respetando las condiciones anteriores, pero si el cambio es inválido, se conserva la programación anterior. Los encuentros finalizados no pueden reprogramarse.
6. Registrar resultados únicamente en encuentros programados y sin resultado previo, con marcadores enteros no negativos. En fútbol sala se permiten empates y se otorgan

3 puntos por victoria, 1 por empate y 0 por derrota. En baloncesto no puede haber empates y se otorgan 2 puntos por victoria y 1 por derrota. Un resultado que sea válido actualiza los puntos y finaliza el encuentro.

7. Mostrar la tabla de posiciones ordenada por puntos de mayor a menor, en caso de empate se ordena por nombre en orden alfabético.

8. Mostrar los encuentros y exhibiciones con programación, ordenados por fecha, hora y nombre de sede, utilizando un único criterio de orden.

9. Mantener el menú disponible hasta que el usuario decida salir, las entradas y operaciones inválidas deben mostrar un mensaje claro, sin que el programa se cierre y sin modificar los datos.

10. Precargar al menos cuatro equipos de dos empresas, cuatro encuentros con ambos deportes representados, dos encuentros con resultado y una exhibición programada, los datos deben incluir un empate en fútbol sala y un desempate por nombre en la tabla.

Regla o comportamiento	**Lo resuelve**	**Por qué**
Fútbol sala: otorga 3 puntos por victoria, 1 por empate y 0 por derrota	Clase concreta	Reglas propias del deporte
Baloncesto: no permite empates, otorga 2 puntos por victoria y 1 por derrota	Clase concreta	Reglas propias del deporte
Compartir equipos, marcador, estados, programación y control del resultado	Clase abstracta	Evita duplicar lo común entre deportes
Permitir programación y aparición en el calendario	Interfaz	Contrato común para encuentros y exhibiciones
Recibir resultados y repartir puntos	Interfaz	Las exhibiciones no tienen resultados
Conservar título, programación y estado de una exhibición	Clase concreta	Solo implementa Programable
Validar conflictos y ofrecer el calendario ordenado	Clase concreta	Conoce todos los equipos y eventos

Acumular puntos y comparar por puntos descendentes y nombre ascendente	Clase concreta	Ordena la tabla sin depender del deporte

Tabla de estados por los que pasa un encuentro

Evento	**Estado actual**	**Operación válida**	**Estado resultante**
Encuentro	No existe	Crear con dos equipos distintos y un deporte.	Sin programar
Encuentro	Sin programar	Asignar fecha, hora y sede válidas, sin conflictos de sede ni de equipos.	Programado
Encuentro	Programado	Reprogramar con datos válidos y sin conflictos.	Programado
Encuentro	Programado	Registrar un resultado válido según el deporte y sumar los puntos de tabla.	Finalizado
Encuentro	Finalizado	No permite reprogramación ni otro resultado.	Finalizado

Tabla de estados por los que pasa una exhibición

Evento	**Estado actual**	**Operación válida**	**Estado resultante**
Exhibición	No existe	Crear con un título.	Sin programar
Exhibición	Sin programar	Asignar fecha, hora y sede válidas, sin conflicto de sede.	Programada
Exhibición	Programada	Reprogramar con datos válidos y sin conflicto de sede.	Programada

Tablas con atributos, métodos, clases, etc.

Tabla de atributos

#	Tipo de dato	Nombre	Visibilidad	Propósito	Clase
1	String	codigo	private	Identificar al equipo, no cambia después de crearlo	Equipo
2	String	nombre	private	Conservar el nombre del equipo	Equipo
3	String	empresa	private	Identificar la empresa representada	Equipo
4	int	puntosTabla	private	Acumular los puntos obtenidos en la copa	Equipo
5	LocalDate	fecha	private	Conservar la fecha del evento	Programacion
6	int	hora	private	Conservar la hora de inicio, entre 0 y 23	Programacion
7	String	sede	private	Conservar el lugar del evento	Programacion
8	Equipo	equipo1	private	Referenciar al primer participante	Encuentro
9	Equipo	equipo2	private	Referenciar al segundo participante	Encuentro
10	Integer	marcador1	private	Guardar el marcador del primer equipo, null mientras no exista resultado	Encuentro
11	Integer	marcador2	private	Guardar el marcador del segundo equipo,	Encuentro

				null mientras no exista resultado	
12	EstadoEvento	estado	private	Indicar si está sin programar, programado o finalizado	Encuentro
13	Programacion	programacion	private	Conservar fecha, hora y sede, inicialmente null	Encuentro
14	String	titulo	private	Describir la exhibición	Exhibicion
15	Programacion	programacion	private	Conservar fecha, hora y sede, inicialmente null	Exhibicion
16	Map<String, Equipo>	equipos	private	Guardar los equipos utilizando su código como clave	Copa
17	Map<String, Programable>	eventos	private	Guardar encuentros y exhibiciones por un identificador de evento	Copa
18	Map<String, Puntuable>	puntuables	private	Localizar los encuentros que admiten resultados, sin verificar el tipo del objeto	Copa

Tabla de métodos con clase, interfaz o clase abstracta

#	**Tipo de retorno**	**Nombre**	**Visibilidad**	**Propósito**	**Clase o interfaz**	**Parámetros**	**Carácter**
1	Programacion	getProgramacion	public	Consultar la programación del evento	Programable	Ninguno	Abstracto
2	EstadoEvento	getEstado	public	Consultar el estado del evento	Programable	Ninguno	Abstracto
3	List<Equipo>	getEquiposOcupados	public	Obtener los equipos cuyo horario ocupa el evento, vacío en exhibiciones	Programable	Ninguno	Abstracto
4	void	programar	public	Asignar o cambiar la programación cuando el estado lo permita	Programable	Programacion nueva	Abstracto
5	int	compareTo	public	Comparar eventos por fecha, hora y sede con un criterio único	Programable	Programable otro	Default, implementa Comparable<Programable>
6	void	registrarResultado	public	Establecer el contrato para recibir resultados y repartir puntos	Puntuable	int marcador1, int marcador2	Abstracto
7	Ninguno	Equipo	public	Inicializar código, nombre y empresa, comenzar con cero puntos	Equipo	String codigo, String nombre, String empresa	Constructor

8	String	getCodigo	public	Consultar el código del equipo	Equipo	Ninguno	Concreto
9	String	getNombre	public	Consultar el nombre del equipo	Equipo	Ninguno	Concreto
10	String	getEmpresa	public	Consultar la empresa representada	Equipo	Ninguno	Concreto
11	int	getPuntosTabla	public	Consultar los puntos acumulados	Equipo	Ninguno	Concreto
12	void	sumarPuntos	De paquete	Acumular una cantidad no negativa de puntos	Equipo	int puntos	Concreto
13	int	compareTo	public	Ordenar por puntos descendentes y después por nombre ascendente	Equipo	Equipo otro	Implementa Comparable<Equipo>
14	Ninguno	Programacion	public	Crear una programación con fecha válida, hora entre 0 y 23 y sede indicada	Programacion	LocalDate fecha, int hora, String sede	Constructor
15	LocalDate	getFecha	public	Consultar la fecha	Programacion	Ninguno	Concreto
16	int	getHora	public	Consultar la hora de inicio	Programacion	Ninguno	Concreto
17	String	getSede	public	Consultar la sede	Programacion	Ninguno	Concreto

18	Ninguno	Encuentro	protecte d	Inicializar dos equipos distintos y establecer el estado sin programar	Encuentro	Equipo equipo1, Equipo equipo2	Constructor de clase abstracta
19	Equipo	getEquipo1	public	Consultar al primer participante	Encuentro	Ninguno	Concreto
20	Equipo	getEquipo2	public	Consultar al segundo participante	Encuentro	Ninguno	Concreto
21	Integer	getMarcador1	public	Consultar el marcador del primer equipo	Encuentro	Ninguno	Concreto
22	Integer	getMarcador2	public	Consultar el marcador del segundo equipo	Encuentro	Ninguno	Concreto
23	EstadoEvento	getEstado	public	Consultar el estado del encuentro	Encuentro	Ninguno	Implementa Programable
24	Programacion	getProgramacion	public	Consultar la programación del encuentro	Encuentro	Ninguno	Implementa Programable
25	List<Equipo>	getEquiposOcupados	public	Devolver una lista no modificable con ambos participantes	Encuentro	Ninguno	Implementa Programable
26	void	programar	public	Asignar la programación e impedir cambios si está finalizado	Encuentro	Programacion nueva	Implementa Programable

27	void	registrarResultado	public	Validar estado y marcador, calcular puntos, guardar el resultado y finalizar	Encuentro	int marcador1, int marcador2	Final, implementa Puntuable
28	void	validarMarcadorDeporte	protected	Establecer la validación particular del deporte	Encuentro	int marcador1, int marcador2	Abstracto
29	int	calcularPuntos	protected	Determinar los puntos de tabla de un participante	Encuentro	int marcadorPropio, int marcadorRival	Abstracto
30	Ninguno	EncuentroFutbolSala	public	Crear un encuentro de fútbol sala llamando al constructor padre	EncuentroFutbolSala	Equipo equipo1, Equipo equipo2	Constructor
31	void	validarMarcadorDeporte	protected	Admitir cualquier marcador no negativo, incluidos los empates	EncuentroFutbolSala	int marcador1, int marcador2	Sobrescrito
32	int	calcularPuntos	protected	Devolver 3 por victoria, 1 por empate o 0 por derrota	EncuentroFutbolSala	int marcadorPropio, int marcadorRival	Sobrescrito
33	Ninguno	EncuentroBaloncesto	public	Crear un encuentro de baloncesto llamando al constructor padre	EncuentroBaloncesto	Equipo equipo1, Equipo equipo2	Constructor
34	void	validarMarcadorDeporte	protected	Rechazar marcadores empatados	EncuentroBaloncesto	int marcador1, int marcador2	Sobrescrito

35	int	calcularPuntos	protected	Devolver 2 por victoria o 1 por derrota	EncuentroBaloncesto	int marcadorPropio, int marcadorRival	Sobrescrito
36	Ninguno	Exhibicion	public	Inicializar el título y dejar la exhibición sin programación.	Exhibicion	String titulo	Constructor
37	String	getTitulo	public	Consultar el título	Exhibicion	Ninguno	Concreto
38	EstadoEvento	getEstado	public	Indicar si está sin programar o programada	Exhibicion	Ninguno	Implementa Programable
39	Programacion	getProgramacion	public	Consultar la programación	Exhibicion	Ninguno	Implementa Programable
40	List<Equipo>	getEquiposOcupados	public	Devolver una lista vacía porque no tiene equipos	Exhibicion	Ninguno	Implementa Programable
41	void	programar	public	Asignar o cambiar la programación	Exhibicion	Programacion nueva	Implementa Programable
42	Ninguno	Copa	public	Inicializar las colecciones vacías	Copa	Ninguno	Constructor
43	void	registrarEquipo	public	Incorporar un equipo y rechazar códigos vacíos o repetidos	Copa	Equipo equipo	Concreto

44	void	registrarEncuentro	public	Registrar un encuentro con equipos de la copa en ambos índices	Copa	String id, Encuentro encuentro	Concreto
45	void	registrarExhibicion	public	Registrar una exhibición únicamente como evento programable	Copa	String id, Exhibicion exhibicion	Concreto
46	Equipo	getEquipo	public	Buscar un equipo por código y rechazar códigos inexistentes	Copa	String codigo	Concreto
47	Programable	getEvento	public	Buscar un evento por identificador y rechazar identificadores inexistentes	Copa	String id	Concreto
48	void	programarEvento	public	Validar conflictos y solicitar la programación o reprogramación	Copa	String id, Programacion nueva	Concreto
49	void	validarConflictos	private	Detectar ocupación de sede o equipos, excluyendo al propio evento	Copa	Programable evento, Programacion nueva	Concreto
50	void	validarIdEvento	private	Rechazar identificadores de evento vacíos o repetidos	Copa	String id	Concreto

51	void	registrarResultado	public	Buscar en puntuables y delegar el registro del resultado	Copa	String id, int marcador1, int marcador2	Concreto
52	List<Equipo>	getTablaPosiciones	public	Devolver una copia no modificable y ordenada de los equipos	Copa	Ninguno	Concreto
53	Iterator<Programable>	iterator	public	Recorrer una copia ordenada y no modificable de los eventos programados	Copa	Ninguno	Implementa Iterable<Programable>
54	void	main	public	Crear la copa, cargar los datos iniciales e iniciar el menú	Main	String[] args	static
55	void	cargarDatosIniciales	private	Registrar los datos de ejemplo mediante las operaciones de la copa	Main	Copa copa	static
56	void	ejecutarMenu	private	Procesar las opciones hasta salir y mostrar errores sin cerrar el programa	Main	Copa copa, Scanner entrada	static
57	int	leerEntero	private	Solicitar un entero y repetir la lectura si el formato es incorrecto	Main	Scanner entrada, String mensaje	static
58	LocalDate	leerFecha	private	Solicitar una fecha y repetir la lectura si es inválida	Main	Scanner entrada, String mensaje	static

Justifique por qué el calendario y la tabla de posiciones se apoyan en dos interfaces distintas en lugar de una sola, y qué pasaría si las exhibiciones tuvieran que cumplir la interfaz de la tabla.

Se utilizan dos interfaces porque programar un evento y registrar un resultado son responsabilidades diferentes. Programable permite que encuentros y exhibiciones aparezcan en el calendario, mientras que Puntuable establece las operaciones para recibir resultados y repartir puntos de tabla, que únicamente corresponden a los encuentros.

Si las exhibiciones tuvieran que implementar Puntuable, estarían obligadas a incluir métodos sin sentido para ellas, como registrar un marcador o repartir puntos, esto es lo que causaría errores innecesarios y otras situaciones.

Explique qué tendría que crear o modificar para agregar un tercer deporte, y si la tabla de posiciones y el calendario tendrían que cambiar

Para agregar un tercer deporte se debe crear una subclase de Encuentro que implemente la validación particular de su marcador y el cálculo de sus puntos de tabla, su constructor llamaría al constructor de la clase abstracta para inicializar los datos compartidos, también habría que agregar al menú la opción de crear encuentros de ese deporte. La tabla de posiciones y el calendario no necesitarían cambios, la tabla seguiría ordenando los puntos acumulados de los equipos, el calendario seguiría utilizando Programable y el mismo criterio de orden de antes.